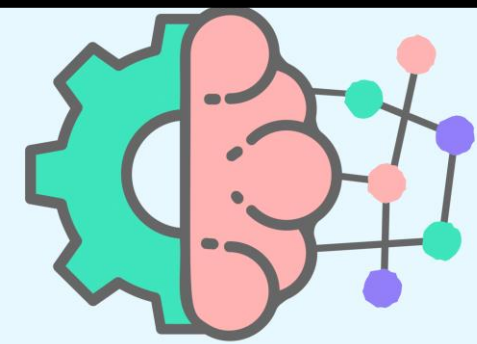


Si nuestro trabajo de CDD apunta a predecir (valores, categorías, valores binarios), tendremos que generar un **Modelo Predictivo**



¿Qué es un Modelo Predictivo?

Un modelo que emplea técnicas de **Machine Learning (ML)**

Un modelo matemático que ha sido diseñado de acuerdo a un algoritmo de ML y entrenado. Puede clasificar, prospectar valores, sintetizar información, o predecir resultados, que no conoce, a partir de nuevos datos

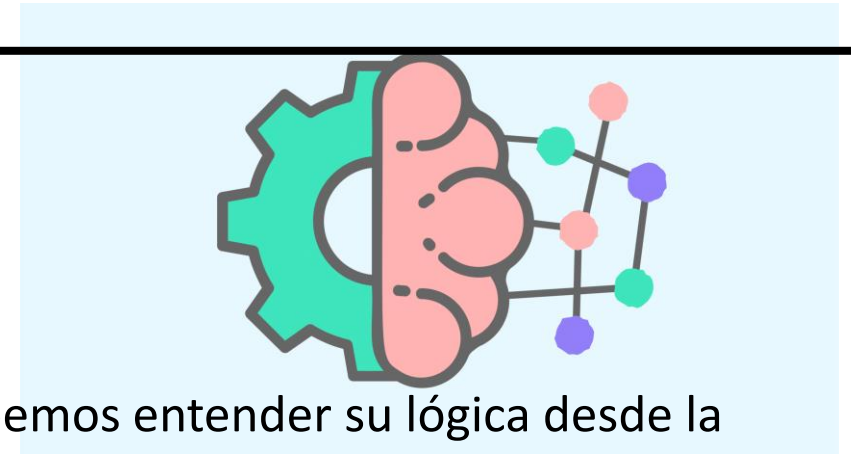
¿Qué es un Algoritmo de ML?

Un conjunto de reglas y procedimientos matemáticos, estadísticos y/o lógicos que un Modelo de Aprendizaje Automático utiliza para comprender patrones y hacer predicciones o juicios basados en datos



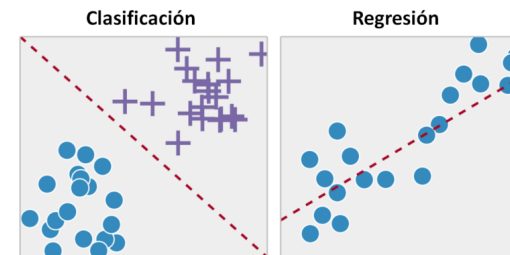
¿Qué tenemos que saber?

- ✓ Cuáles son los algoritmos de ML que tenemos a disposición
- ✓ Cuándo es viable y conveniente aplicar cada uno de ellos. Debemos entender su lógica desde la abstracción. No necesitamos conocer cómo implementarlos; para eso hay funciones y herramientas de todo tipo. Cuando todo falla: la IA amiga puede salir al rescate (sólo hay que saber promptear)
- ✓ Cómo armar un Modelo y usar un Algoritmo de ML



Ej:

Si tenemos una estructura de datos y deseamos, por alguna razón, accederlos ordenadamente sabemos que debemos ordenarlos (en Python tenemos `sort()`, `sorted()` y todas las herramientas de ordenamiento que haya en los módulos disponibles). No necesitamos escribir el algoritmo de ordenamiento de un método específico!!!



Los Algoritmos de ML se clasifican en tres tipos

Aprendizaje Supervisado

Los métodos de Aprendizaje Supervisado tienen el objetivo de predecir una variable respuesta Y a partir de una serie de predictores

Para ello, se dispone de p características ($X_1, X_2 \dots X_p$) y de la variable respuesta Y medidas en n observaciones

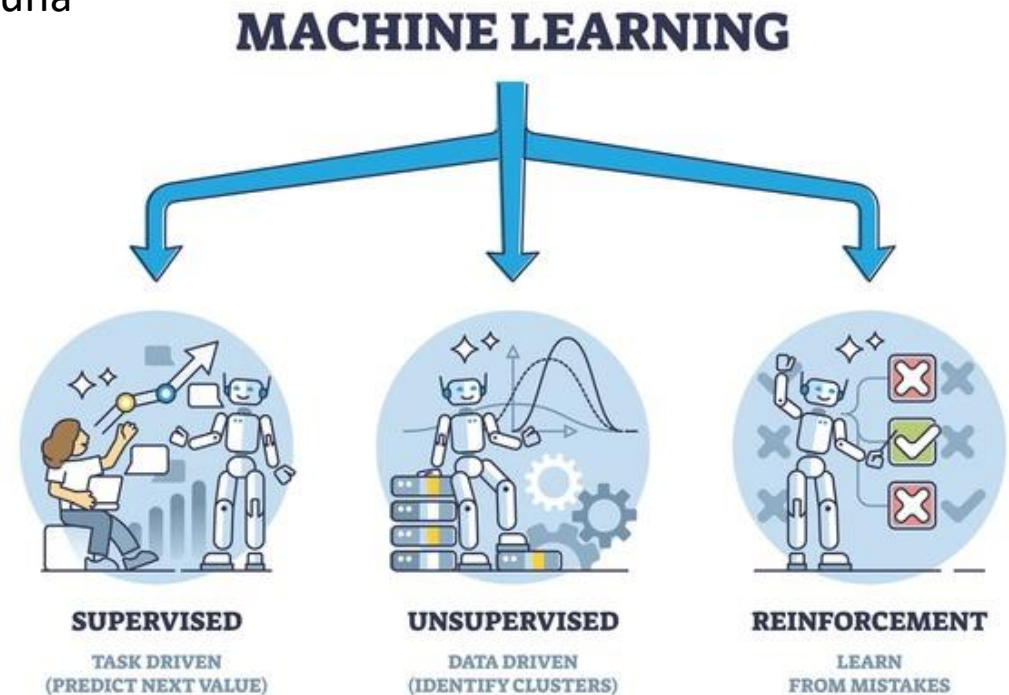
Aprendizaje no Supervisado

La variable respuesta Y no se tiene en cuenta ya que el objetivo no es predecir Y sino extraer información empleando los predictores, por ejemplo, para identificar subgrupos

Aprendizaje por Refuerzo

Se basa en el proceso de aprendizaje por prueba y error que utilizan los humanos

1. El agente realiza una acción que cambia el estado del entorno
2. El agente recibe una recompensa o penalización por la acción realizada



Tipos de Modelos de ML

Aprendizaje Supervisado

K-NN
Regresión Lineal
Regresión Logística
Árboles de Decisión
PLS
Bosque Aleatorio
Naive Bayes
Redes Neuronales

Aprendizaje por Refuerzo

Q-Learning
DQN
SARSA

Aprendizaje No Supervisado

K-means
DBSCAN
PCA
SNE
t-SNE
LOF

Aprendizaje Supervisado

Aprendizaje no Supervisado

Aprendizaje por Refuerzo



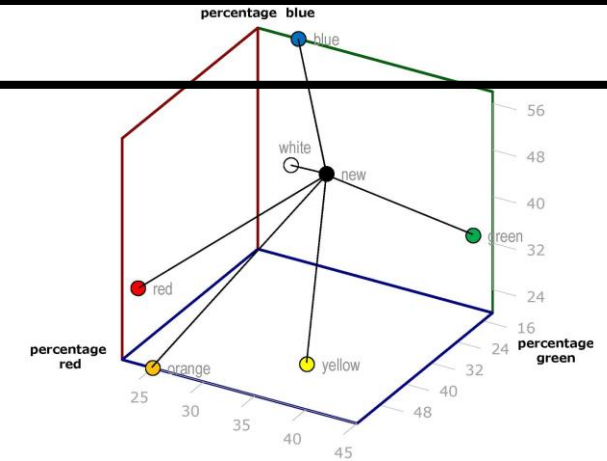
Modelos de aprendizaje supervisado

El modelo de aprendizaje supervisado es una categoría particular de diferentes modelos de ML que utilizan **datos etiquetados** para el entrenamiento. Los casos de entrenamiento vienen con sus respuestas

El algoritmo aprende a generar predicciones o juicios al mapear los datos de entrada a **etiquetas objetivo** conocidas

K-Vecinos más cercanos (K-NN)

Algoritmo básico, pero efectivo, de clasificación y regresión
Determina la clase o el valor de tus datos según la opinión de la mayoría de tus k-vecinos más cercanos (o el promedio de sus opiniones) en los datos de entrenamiento



K-Vecinos más cercanos (K-NN)

El algoritmo k-vecinos más cercanos (kNN) funciona de la siguiente manera:

- Calcula la distancia entre el elemento a clasificar y el resto de elementos del conjunto de entrenamiento
- Selecciona los **k** elementos más cercanos
- Realiza una **votación de mayoría** entre los **k** puntos (filas). Se lo etiqueta con la clase/etiqueta que **gana**

*El valor de **k** define cuántos vecinos votarán*

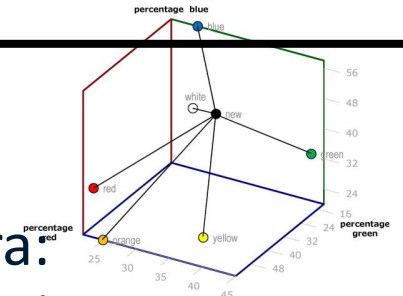
Es uno de los algoritmos de clasificación y regresión más populares y simples que se utilizan en la actualidad

*Se usa comúnmente para sistemas de recomendación simples, reconocimiento de patrones, predicciones de mercados financieros, detección de intrusiones y más (en general, **clasificaciones o regresiones**)*

Tips:

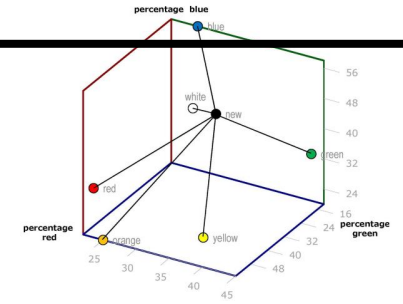
Con muchos datos (filas y columnas) K-NN se vuelve cada vez más ineficiente

Con un buen k no es muy sensible a outliers



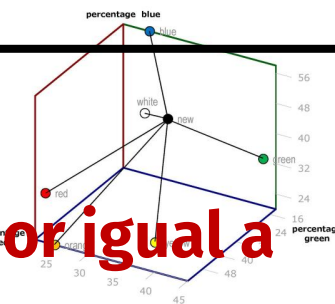
Pasos del Algoritmo K-NN

- ✓ Elegir el número de vecinos (k). Decidir cuántos vecinos más cercanos a considerar
- ✓ Calcular la distancia. Para un nuevo punto de entrada, se calcula la distancia entre este punto y todos los puntos del conjunto de entrenamiento. Las métricas de distancia comunes son: Euclidiana (más usada), Manhattan, Coseno, Minkowski
- ✓ Identificar los k vecinos más cercanos. Se seleccionan los k puntos con menor distancia al nuevo
- ✓ Votar o promediar. Se elige la clase que más se repite entre los k vecinos (votación mayoritaria) o se calcula el promedio (o mediana) de los valores de salida de los k vecinos
- ✓ Asignar la etiqueta o valor. Se asigna al nuevo punto la clase o valor obtenido del paso anterior



Tip:

Conviene normalizar las características para que todas contribuyan por igual a la distancia



Ej: Clasificar un nuevo punto usando k-NN con $k = 3$.

Datos de entrenamiento:

Punto	X	Y	Clase
A	1	2	Rojo
B	2	3	Rojo
C	3	3	Azul
D	6	5	Azul
E	7	7	Azul

Nuevo punto a clasificar:

Punto	X	Y	Clase
?	3	4	¿?

Paso 1: Elegir $k = 3$

Queremos encontrar los **3** vecinos más cercanos

Paso 2: Calcular distancias al nuevo punto (3, 4)

Usamos distancia Euclidiana:

$$A (1,2): \sqrt{(3-1)^2 + (4-2)^2} = \sqrt{(4 + 4)} = 2,83$$

$$B (2,3): \sqrt{(3-2)^2 + (4-3)^2} = \sqrt{(1 + 1)} = 1,41$$

$$C (3,3): \sqrt{(3-3)^2 + (4-3)^2} = \sqrt{(0 + 1)} = 1,00$$

$$D (6,5): \sqrt{(3-6)^2 + (4-5)^2} = \sqrt{(9 + 1)} = 3,16$$

$$E (7,7): \sqrt{(3-7)^2 + (4-7)^2} = \sqrt{(16 + 9)} = 5,00$$

Paso 3: Elegir los 3 vecinos más cercanos

Los 3 con menor distancia:

1. C (3,3) – Azul – Distancia: 1,00
2. B (2,3) – Rojo – Distancia: 1,41
3. A (1,2) – Rojo – Distancia: 2,83

Paso 4: Votar entre las clases

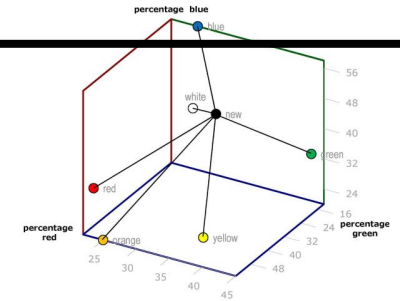
Rojo: 2 vecinos (A, B)

Azul: 1 vecino (C)

Paso 5: Clasificación final

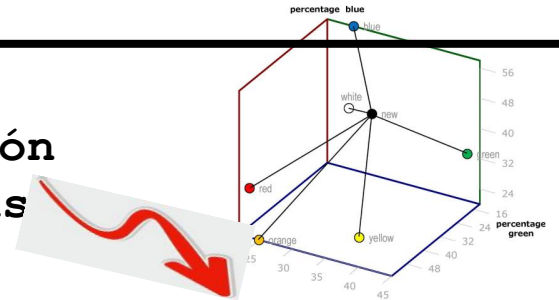


El nuevo punto (3,4) se clasifica como **Rojo**



Python

```
from sklearn.neighbors import KNeighborsClassifier # por votación
# Datos de entrenamiento (solo las coordenadas), Características
X = [
    [1, 2], # A
    [2, 3], # B
    [3, 3], # C
    [6, 5], # D
    [7, 7]  # E
]
# Etiquetas (clases) correspondientes, Objetivo
y = ['Rojo', 'Rojo', 'Azul', 'Azul', 'Azul']
# Nuevo punto a clasificar
nuevoPunto = [[3, 4]] # fíjate: lista de lista, podrían ser más de uno
# Crear y entrenar el modelo k-NN
k = 3
modelo = KNeighborsClassifier(n_neighbors=k)
modelo.fit(X, y)
# Hacer la predicción
prediccion = modelo.predict(nuevoPunto)
# Resultado
print(f"El nuevo punto {nuevoPunto[0]} se clasifica como: {prediccion[0]}")
```



👉 # si se quiere usar el promedio
importar **KNeighborsRegressor**
Con `weights='distance'`, los vecinos
más cercanos tienen más influencia

👉 El nuevo punto (3,4) se clasifica como **Rojo**

Python

¿Por qué elegir bien k ?

Un k muy bajo (ej: 1) puede sobreajustar (sensible al conjunto de training seleccionado)

Un k muy alto puede subajustar (demasiado general)

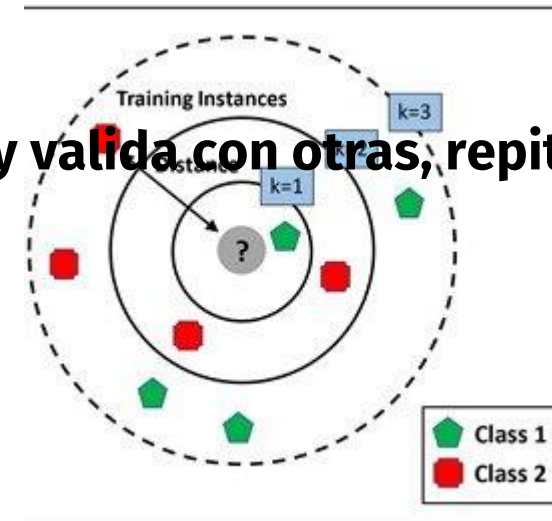
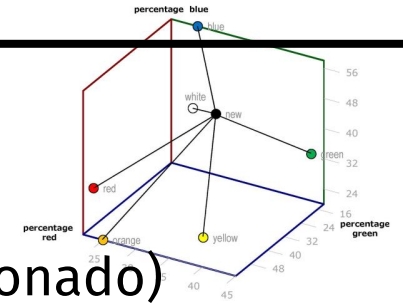
Elegir un k que tenga el mejor desempeño general

¿Cual k ? si hay dos ganadores ¿Qué hacemos? Un $k=3$ gana ahí

Mejor k no tan bajos y no tan altos suelen estar entre **3-5**

Se puede usar validación cruzada (cross-validation)

Este método divide el dataset en partes, entrena con alguna y valida con otras, repitiendo el proceso. Luego promedia el rendimiento

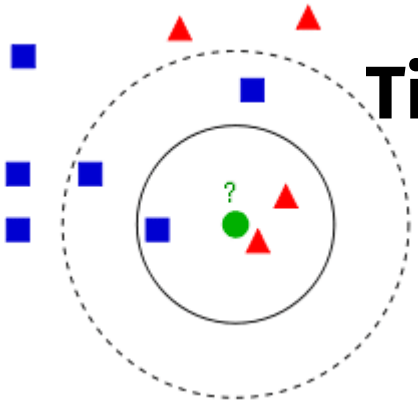
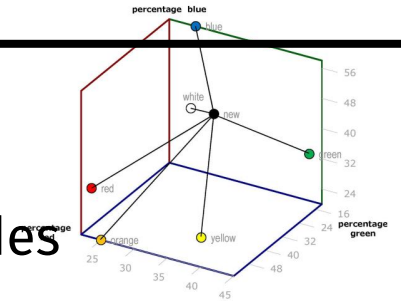


¿Qué pasa cuando ingreso datos para predicción en k-NN?

En k-NN, el método **fit()** **no** entrena en el sentido tradicional (como sí lo hacen redes neuronales, árboles de decisión, etc.).

En realidad, cuando se invoca `modelo.fit(X_train, y_train)` no hay un entrenamiento en el sentido tradicional. El modelo simplemente almacena los datos de entrenamiento para usarlos más adelante en la predicción. No ajusta parámetros ni encuentra una fórmula. Sólo nos evitará volver a indicarle k, el método de donación, etc.

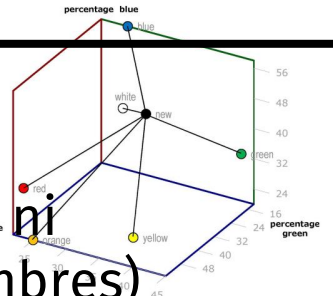
Así que cuando se invoca `predict(nuevo_dato)`... Se accede a los datos guardados del entrenamiento (`X_train, y_train`), calcula la distancia del nuevo dato a **cada punto** de `X_train` (por suerte es sólo de ese dato!!!), ordena por distancia y elige los k más cercanos, clasifica o predice en base a esos vecinos.



Tiene que recalcular distancias cada vez que predice

¿Qué pasa cuando ingreso datos para predicción en k-NN?

- ✓ **Es muy barato al entrenar** (no hace nada complejo, no arma una función de ajuste, ni una estructura decisional, ni tienen que volver atrás recalculando pesos y certidumbres)
- ✓ **Es costoso al predecir** (sobre todo con muchos datos, muchas dimensiones, o ambos)

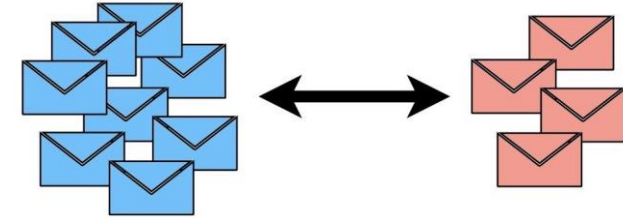


¿Por qué es tan popular?

- ✓ Es muy simple de implementar
- ✓ Funciona bien en problemas no lineales (todo lo que no va con regresión, otro método simple)
- ✓ No necesita ajustar parámetros internos
- ✓ Es útil como modelo de partida para comparar modelos

Optimización posible:

Librerías como **scikit-learn** usan estructuras como **KD-Trees** o **Ball Trees** internamente para acelerar la búsqueda de vecinos. Pero incluso con estas estructuras, se sigue calculando distancias, solo que de forma más eficiente

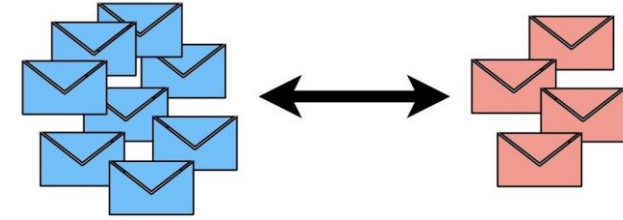


Naive Bayes

Algoritmo de clasificación probabilística basado en el teorema de Bayes (Bayes) y en la concepción de que las características son independientes (Naive)

Realiza tareas de categorización de texto, como la detección de spam y el análisis de sentimientos. En general es bueno para clasificar en dos categorías disjuntas

Aunque no es bueno manejando características correlacionadas, cosa que casi siempre aparece..., igual es bueno en clasificaciones binarias



Pasos del algoritmo Naive Bayes

Entrenamiento:

Se calculan las probabilidades totales para cada clase, $P(\mathbf{C})$ (prevalencias)

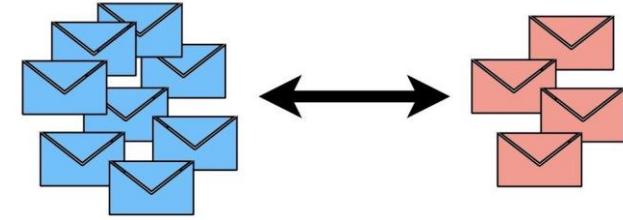
Se calculan las probabilidades condicionales para cada columna ($\mathbf{x}_i$) dado cada clase $\mathbf{C}$

¿Cómo?

- ✓ Si los atributos son categóricos, se cuenta frecuencia.
- ✓ Si son continuos, se suele asumir una distribución teórica, como la normal u otra y se estima la media y desviación estándar

Predicción:

- ✓ Para un nuevo caso $\mathbf{X}=(\mathbf{x}_1,\mathbf{x}_2,...,\mathbf{x}_n)$, se calcula Bayes para cada clase $\mathbf{C}$
- ✓ Se escoge la clase con la mayor probabilidad



Pasos del algoritmo Naive Bayes

Ejemplo categórico (MultinomialNB)

Queremos predecir si alguien va a jugar al tenis según el clima:

Día	Humedad	Juega?
Soleado	Alta	No
Soleado	Alta	No
Nublado	Alta	Si
Lluvioso	Moderada	Si
Soleado	Moderada	Si

Queremos predecir si se juega al tenis si el día está **Soleado** y la humedad es **Moderada**

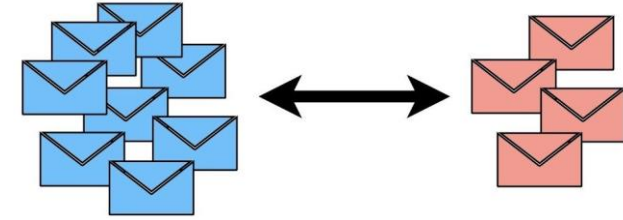
Usamos frecuencias para estimar probabilidades y luego aplicamos el teorema para ambas clases

Dependiendo de los tipos de atributos convendrá usar el cálculo de probabilidades con:

GaussianNB (continuos, asumiendo normalidad, `sklearn.naive_bayes.GaussianNB`)

MultinomialNB (categóricos, típico en texto, `sklearn.naive_bayes.MultinomialNB`)

BernoulliNB (binarios, presencia/ausencia, `sklearn.naive_bayes.BernoulliNB`)



Algoritmo Naive Bayes

Ventajas

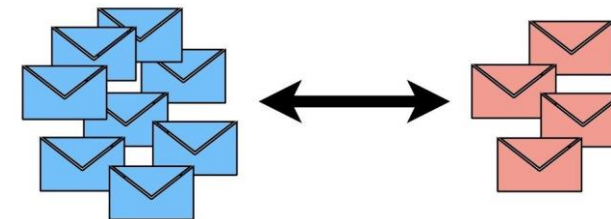
- ✓ Rápido y eficiente, incluso con muchos atributos
- ✓ Requiere pocos datos para entrenarse bien

Desventajas

- ✓ Supone independencia entre atributos (casi nunca se cumple); no maneja bien características correlacionadas

Conclusión:

Buena respuesta, incluso con características correlacionadas



Algoritmo Naive Bayes

Ejemplo: Naives_Bayes con Python usando Base de Datos Iris

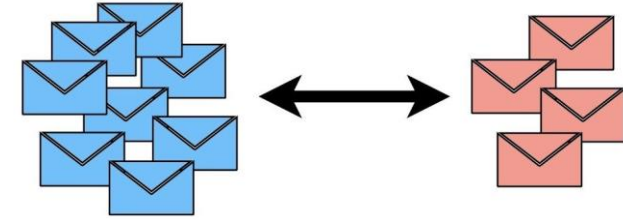
Este dataset incluye 150 flores con 4 características numéricas reales:

- Largo del sépalo
- Ancho del sépalo
- Largo del pétalo
- Ancho del pétalo

Y 3 posibles clases:

- setosa
- versicolor
- virginica





Algoritmo Naive Bayes

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB #las colum son num reales
from sklearn.metrics import accuracy_score #para evaluar resultados
```

```
iris = load_iris() # Cargamos el dataset Iris
X = iris.data
y = iris.target
```

👉 Precisión del modelo Naive Bayes: 0.98

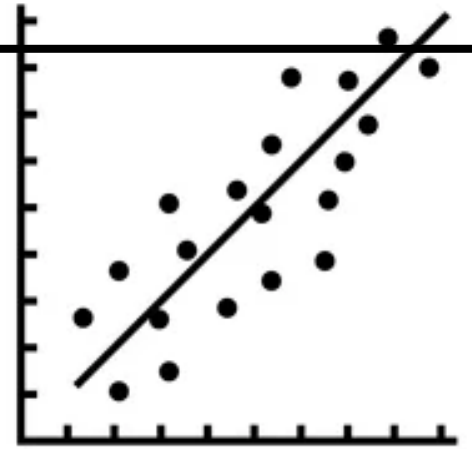
```
# Dividimos en entrenamiento y prueba, acá si hay entrenamiento real
XEntr, XTesteo, yEntr, yTesteo = train_test_split(X, y, test_size=0.3,
                                                  random_state=42)
```

```
modelo = GaussianNB() # Creamos el modelo Naive Bayes usando normal
modelo.fit(XEntr, yEntr) # Entrenamos el modelo
yPred = modelo.predict(XTesteo) # Predecimos sobre el set de prueba
precision = accuracy_score(yTesteo, yPred) # Medimos la precisión
print(f"Precisión del modelo Naive Bayes: {precision:.2f}")
```

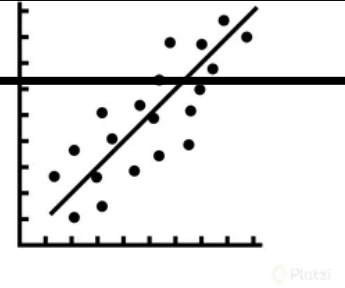
Regresión lineal

Los modelos que aplican regresión lineal predicen una salida numérica continua

Identifica la mejor conexión lineal (o ajuste) entre la o las variables de entrada (características, variables independientes) y la variable objetivo (variable dependiente)



La regresión lineal es una técnica estadística que se utiliza para modelar y analizar la relación entre una variable dependiente y una o más variables independientes. Se basa en la idea de que la variable desconocida o dependiente se puede predecir a partir de valores de datos relacionados y conocidos



Regresión lineal

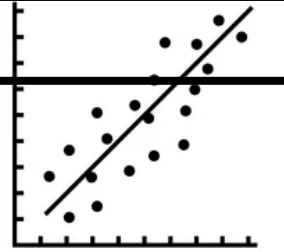
Ventajas:

- ✓ Es uno de los algoritmos más simples y eficientes de Aprendizaje Supervisado
- ✓ Es muy sencilla de modelar e interpretar
- ✓ Puede ser simple (se estudia el efecto en la variable dependiente de una sola variable independiente), o múltiple (el efecto combinado de varias variables)
- ✓ Sirve como modelo de base para comparar con otros

Desventajas:

- ✓ Asume una relación lineal entre las variables, en otros casos no es bueno
- ✓ Sensible a valores atípicos (outliers)

Se puede aplicar **PCA** antes de Regresión para reducir dimensionalidad y luego aplicar un modelo de RL a las super columnas. Esta técnica es **PCR** o **PLS** , que pertenece a la categoría de Aprendizaje Supervisado



Tipos de RL

Regresión lineal simple: Cuando hay una sola variable independiente

$$y=wx+b$$

y: variable dependiente (lo que se quiere predecir)

x: variable independiente (la entrada)

w: pendiente o coeficiente de regresión (cuánto cambia y por cada unidad que cambia x)

b: término de sesgo o intersección (donde la línea cruza el eje y)

Regresión lineal múltiple: Cuando hay más de una variable independiente

Idem fórmula anterior, pero ahora w y x son vectores

¿Cómo funciona la RL?

Entrenamiento del modelo:

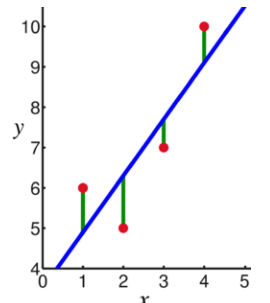
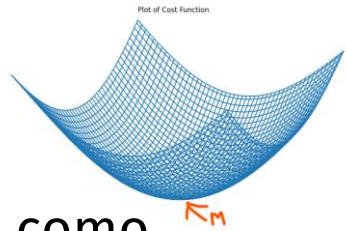
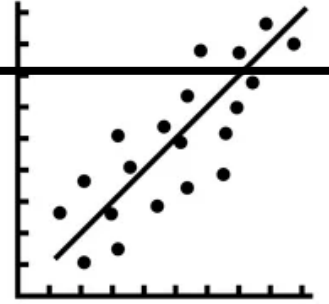
El modelo aprende los valores óptimos de los coeficientes w y el término b a partir de un conjunto de datos etiquetado (con valores reales de y). Utiliza una función de pérdida, comúnmente el Error Cuadrático Medio MSE donde se promedian los desvíos entre el valor y real y el predicho por el modelo

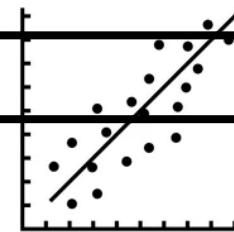
Optimización:

Para minimizar el MSE y encontrar los mejores w y b , se puede usar solución analítica como Mínimos Cuadrados o un Algoritmos de Optimización, como el Descenso de Gradiente

Predicción:

Una vez entrenado, el modelo puede predecir nuevos valores de y para nuevas entradas x , simplemente aplicando la fórmula aprendida





¿Cómo funciona la RL?

Ejemplo con Python: Vamos a predecir el puntaje en un examen en base a las horas estudiadas

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

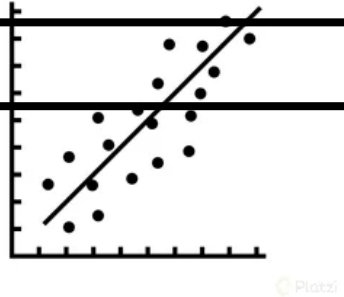
# Datos simulados x:número de horas estudiadas y:puntaje en el examen
x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9]).reshape(-1, 1)
y = np.array([35, 45, 50, 55, 60, 65, 70, 75, 80])

# Dividimos en entrenamiento y prueba
xEn, xTst, yEn, yTst = train_test_split(x, y, test_size=0.2, random_state=42)

modelo = LinearRegression() # Creamos el modelo y lo entrenamos
modelo.fit(xEn, yEn)

yPred = modelo.predict(xTst) # Predecimos valores
```

👉 Código de salida en Próxima Diapo

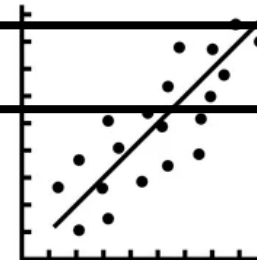


¿Cómo funciona la RL?

```
# Mostramos coeficientes y término independiente calculados
print(f"Coeficiente (pendiente): {modelo.coef_[0]}")
print(f"Término independiente (intercepto): {modelo.intercept_}")

# Graficamos resultados
plt.scatter(x, y, color="blue", label="Datos reales")
plt.plot(x, modelo.predict(x), color="red", label="Línea de regresión")
plt.xlabel("Horas estudiadas")
plt.ylabel("Puntaje en el examen")
plt.title("Regresión Lineal Simple")
plt.legend()
plt.grid(True)
plt.show()
```

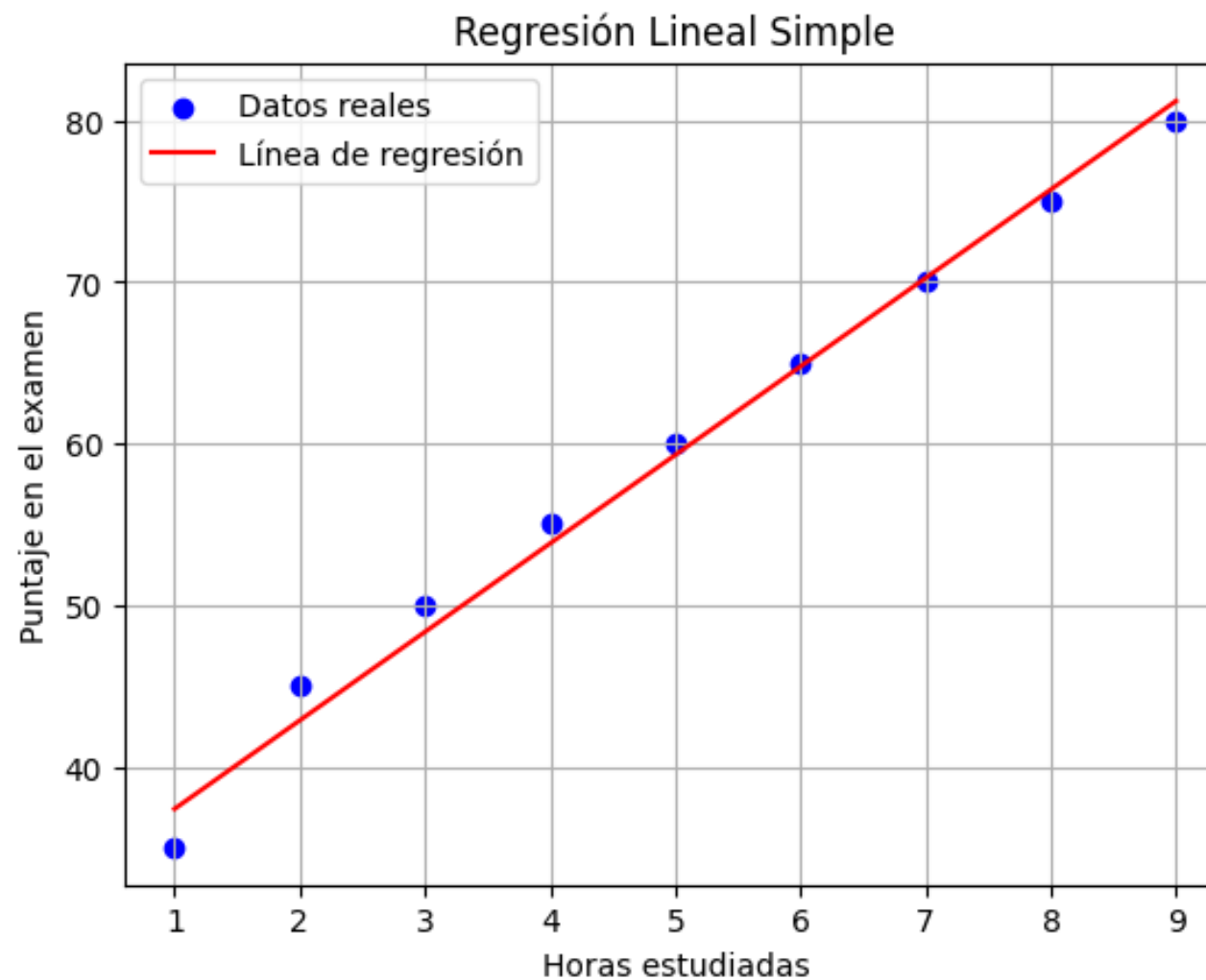
👉 Salida en Próxima Diapo

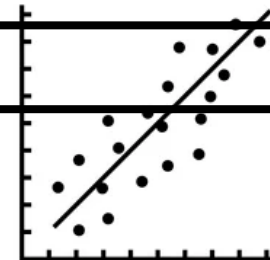


¿Cómo funciona la RL?

Coeficiente (pendiente): 5.476190476190474

Término independiente (intercepto): 31.904761904761916





¿Cómo medir la Precisión del Modelo?

Se usa alguna de las siguientes métricas:

MSE: Error cuadrático medio

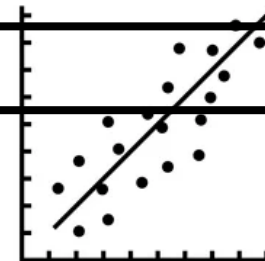
RMSE: Raíz del error cuadrático medio

MAE: Error absoluto medio

R²: Coeficiente de determinación (qué tan bien se ajusta el modelo)

Interpretación de cada una:

MSE	Penaliza más los errores grandes. Cuanto menor, mejor
RMSE	Similar al MSE pero en la misma escala que la variable de salida
MAE	Promedio de errores absolutos, más robusto ante outliers que el MSE
R²	Cuánto del total de la variación de los datos explica el modelo (1.0 es perfecto, 0 es aleatorio)

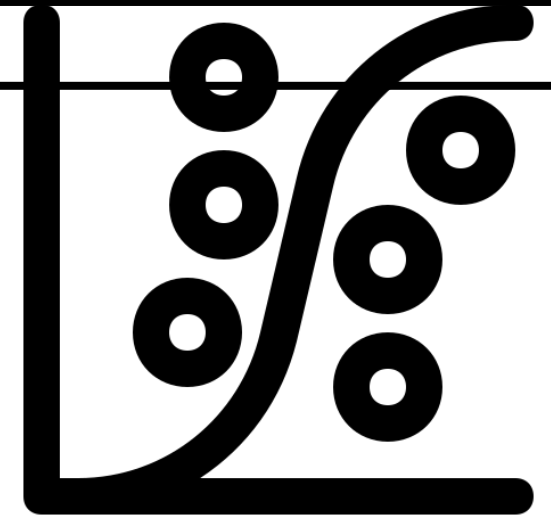


¿Cómo medir la Precisión del Modelo?

Medida	Cuándo usarla	Qué te dice
MAE	Cuando todos los errores te importan igual	Error promedio (en las mismas unidades)
MSE	Cuando quieres castigar más los errores grandes	Error cuadrático (sin unidades claras)
RMSE	Cuando quieres penalizar errores grandes, pero mantener interpretabilidad	Error medio (en las mismas unidades)
R^2	Cuando quieres medir la “capacidad explicativa” del modelo	Qué tanto explica el modelo de la variación real

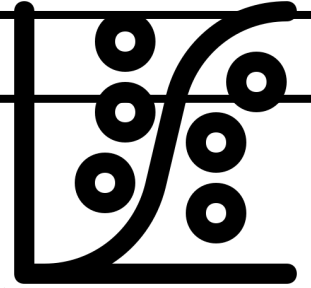
MSE	Predecir el costo de un tratamiento médico . Fallar por \$1000 es mil veces peor que fallar por \$1, quiero que el modelo evite errores grandes.
RMSE	Cuando quiero el mismo efecto anterior pero medido en la moneda en que se expresan los costos.
MAE	Tengo que predecir tiempo de entrega de paquetes en días. Me da igual equivocarme por 1 día o por 2 (todos los errores cuestan lo mismo)
R^2	Indicador global fácil de entender (“mi modelo explica el n% de la realidad, el resto son variaciones que no están capturadas en el modelo”)

Regresión logística



Es un algoritmo de Clasificación que se utiliza para tareas de clasificación binaria (con una elección binaria: sí/no, como resultado)

Basado en los atributos de entrada, calcula la probabilidad de un resultado binario. El nombre de Regresión Logística proviene de la función base (recta) que usa el algoritmo para ajustar los coeficientes w y el término b , pero usando probabilidades



RLog, cuándo usarla

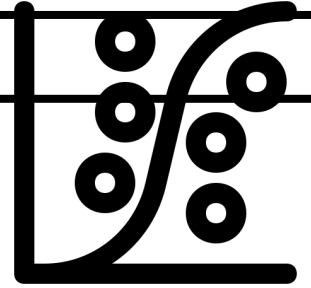
- ✓ La variable a predecir es categórica binaria (0 o 1, Verdadero o Falso, extranjero, local, mayor de edad, menor de edad, etc.
- ✓ Se quiere obtener probabilidades de pertenencia a una clase

Pasos del algoritmo

- ✓ Modelo base: función lineal $y = w^T x + b$
- ✓ Activación: para convertir ese valor y en una probabilidad entre 0 y 1, se pasa por la función **sigmoide**
- ✓ Clasificación: se aplica un umbral para convertir la probabilidad en clase
- ✓ Si el valor está por debajo del umbral se clasifica como de una clase
- ✓ (por ejemplo falso) y si está por arriba como la otra (verdadero)

El umbral puede ajustarse según el problema.





Pasos del algoritmo

Entrenamiento (Ajuste de Pesos)

Se utiliza una función de cálculo de pérdida llamada log-loss o cross-entropy y se optimiza esta pérdida usando Descenso de Gradiente para ajustar los parámetros w y b de la ecuación lineal (es un ajuste lineal de pesos o probabilidades)

¿Qué devuelve el modelo?

Para cada caso una probabilidad de que pertenezca a la Clase 1

O...

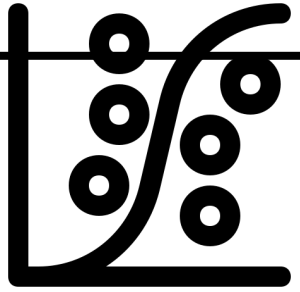
Una predicción binaria (pertenece a la Clase 1 o a la Clase 2), si se aplica un umbral

Ejemplos comunes:

¿Un correo es spam o no?

¿Un paciente tiene diabetes o no?

¿Un cliente comprará o no?

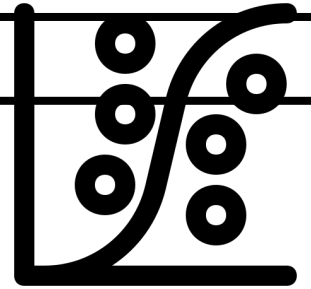


Ejemplo en Python: Predecir si un estudiante aprobará un examen en base a horas de estudio

```
import random
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Genera datos aleatorios (100 estudiantes)
random.seed(42)
X = [] #tabla; lista de listas
y = [] #lista simple
for _ in range(100):
    horas=random.randint(0,10)
    probAprob= horas/10 # más estudia, más probabilidad
    aprueba= 1 if random.random() < probAprob else 0
    X.append([horas]) # scikit-learn espera tablas (n colum)
    y.append(aprueba)
```

👉 Sigue en Próxima Diapo



Ejemplo en Python

Divide en entrenamiento y prueba

```
XEnt, Xtst, yEnt, yTst = train_test_split(X, y, test_size=0.3,  
                                          random_state=42)
```

Entrena el modelo

```
modelo = LogisticRegression()  
modelo.fit(XEnt, yEnt)
```

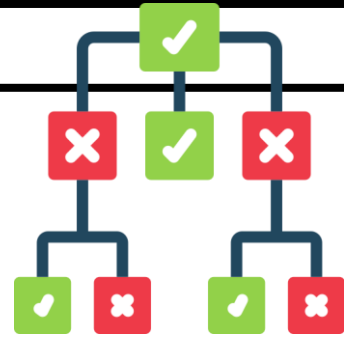
Predicción

```
yPred = modelo.predict(XTst)
```

Evalúa

```
print("Exactitud del modelo:", accuracy_score(yTst, yPred))
```

👉 Exactitud del modelo: 0.9



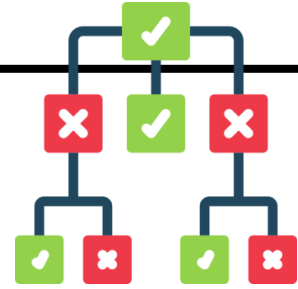
Árboles de decisión

Se utilizan para modelos de clasificación y regresión

Construyen una estructura similar a un árbol donde cada nodo refleja una decisión basada en una característica, y las hojas representan una etiqueta de clase final o un valor numérico

Bosque aleatorio

Estrategia de aprendizaje por conjuntos que combina numerosos árboles de decisión para aumentar la precisión de las predicciones y reducir el **sobreajuste**



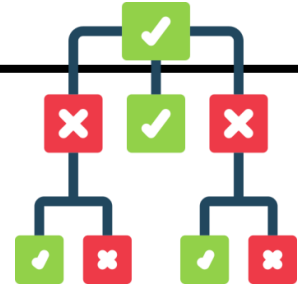
Árboles de decisión (AD) ¿Qué son y cómo funcionan?

Es un modelo que divide los datos en partes más pequeñas, como si hiciéramos preguntas tipo **sí/no** (enigmáticos) en cada paso, hasta llegar a una decisión final

Construcción del Modelo:

1. **Selecciona la mejor variable (característica/columna).** El algoritmo busca la mejor pregunta que divide los datos lo más claramente posible según la variable objetivo. Su objetivo es **maximizar la pureza**
2. **Divide los datos.** Los datos se dividen en dos grupos según la pregunta (como si fueran ramas del árbol)
3. **Repite recursivamente.** Se repite el proceso en cada subconjunto. El árbol crece en profundidad
4. **Detenerse.** El árbol para de crecer cuando se da alguna de las siguientes condiciones:
 - ✓ Ya no hay más ganancias al dividir
 - ✓ Se alcanza una profundidad máxima
 - ✓ Quedan pocos datos en una rama
 - ✓ Todos los datos en una rama tienen la misma clase (clasificación)

👉 **Mucha tela para cortar...**



Árboles de decisión ¿Qué son y cómo funcionan?

Paso 1

¿Cómo elige la **mejor** pregunta? Mide qué tan mezclados están los datos en una división del árbol de decisión(esto lo hará por cada columna que le falte colocar de nodo...)

Si todos los datos en una hoja son de la misma clase, está pura (perfecta)

Si hay una mezcla de clases, está impura (mala división)

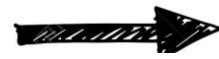
¿Y cómo sabe el grado de pureza?

Para clasificación: Aplica **Gini** o **Entropía** y busca la división que reduzca más la impureza

Para regresión: Usa métricas como Error cuadrático medio, **MSE** o Reducción de varianza

Si tienes dos grupos de 10 amigos

Grupo 1: 5 prefieren el fernet y 5 el gin



Muy Impuro (totalmente mezclados)

Grupo 2: 9 prefieren el fernet y 1 el gin



Más pureza

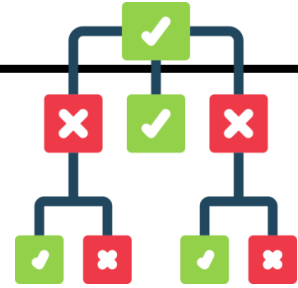
¿Qué busca **Gini**? (default en **scikit-learn** pero puedes cambiarlo a método **entropy**)

Gini mide la probabilidad de que dos amigos elegidos al azar tengan una preferencia distinta. Si todos eligen lo mismo es máxima pureza, Gini 0; si están 50/50, Gini 0.5

Grupo 1 Gini ≈ 0.5

Grupo 2 Gini ≈ 0.18

 **El Algoritmo elegirá como nodo del árbol al Grupo 2**



Árboles de decisión ¿Qué son y cómo funcionan?

Paso 1

¿Qué busca Entropía?

Entropía es un concepto de Teoría de Sistemas que mide el desorden o la sorpresa

Si todos son iguales, **Entropía = 0**



sin sorpresa

Si están 50/50. **Entropía = 1**



máximo desorden

Grupo 1, Entropía ≈ 1.0

Grupo 2, Entropía ≈ 0.47

Gini es más veloz



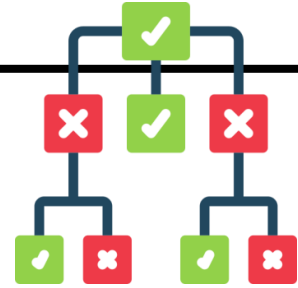
Entropía es más balanceado

Paso 2 y 3

Se pueden usar árboles no binarios, pero no es común porque **Donde manda capitán, no manda marinero ...**

Los árboles B invertidos son estructuras de datos recontra usadas en Software y cualquier nodo que ramifique con más de dos ramas puede ser representado por un subárbol equivalente ...

Los mejores algoritmos de generación y recorrida de árboles B son recursivos



Árboles de decisión ¿Qué son y cómo funcionan?

Paso 4

¿Cuándo se detiene un árbol de decisión? Cuando se da alguna de las siguientes situaciones

- ✓ Todos los datos en el nodo son del mismo tipo o clase
- ✓ Ya no quedan más características para dividir
- ✓ El nodo tiene muy pocos datos (ej: sólo tiene asociado 5 filas)
- ✓ Se alcanzó una profundidad máxima del árbol. Se puede poner un límite al número de divisiones

¿Qué devuelve el árbol?

En clasificación: **la clase más común en la hoja (DecisionTreeClassifier)**

En regresión: **el valor promedio en la hoja (DecisionTreeRegressor)**

Ventajas:

Muy fácil de entender e interpretar

No necesita normalizar datos

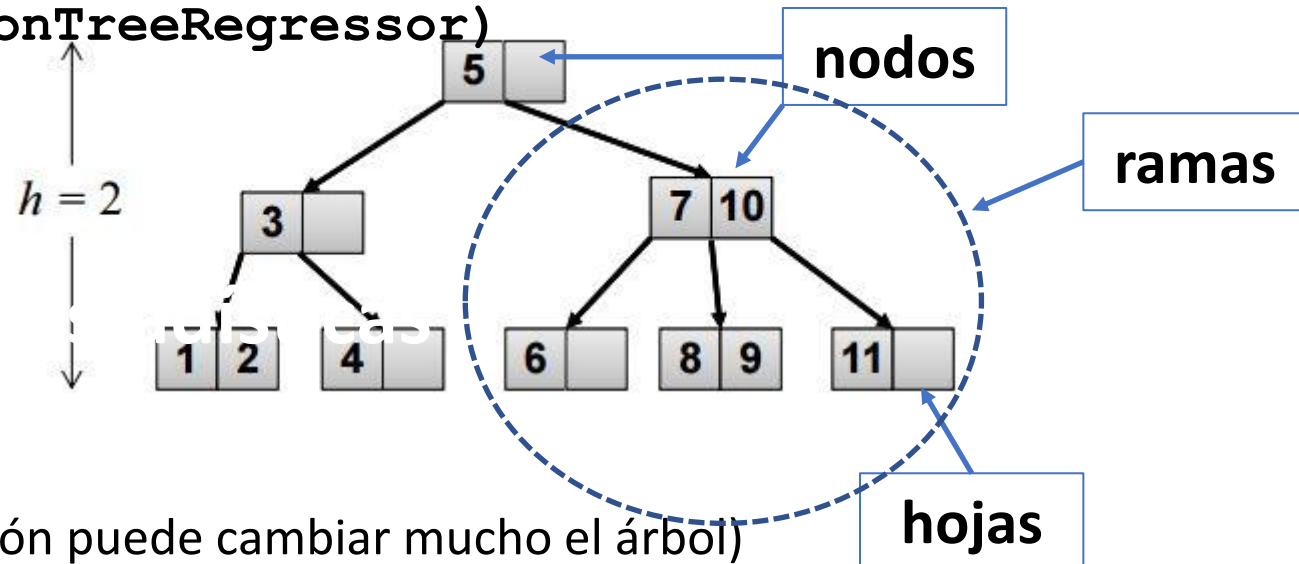
Funciona con variables categóricas y numéricas

Puede capturar relaciones no lineales

Desventajas:

Tiende a sobreajustarse (overfitting) si no se limita

Sensible a cambios en los datos (una pequeña variación puede cambiar mucho el árbol)



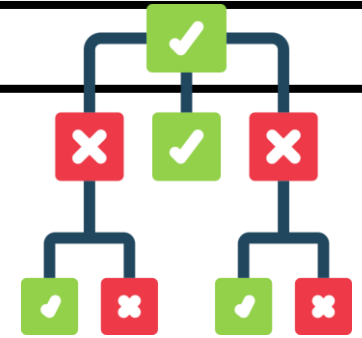
Árboles de decisión

Ejemplo Python: Predecir si una persona comprará o no un producto según edad e ingresos

```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Dataset simple (Edad, Ingresos) – ingresos en miles
X = [[25, 40], [30, 20], [45, 80],
     [35, 60], [50, 90], [23, 25],
     [40, 100], [60, 70], [22, 15],
     [48, 85]]

y = [0, 0, 1, 1, 1, 0, 1, 1, 0, 1] # 0 = No compra, 1 = Compra
```



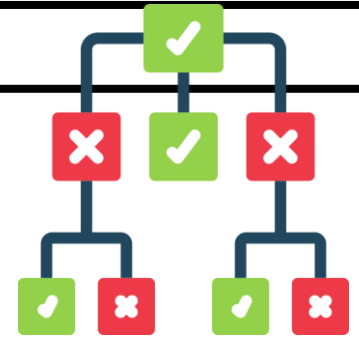
Árboles de decisión

```
XEnt, XTst, yEnt, yTst = train_test_split(X, y, test_size=0.3,  
                                           random_state=42)
```

```
# Arma modelo, máxima profundidad en 3 y entrena  
modelo = DecisionTreeClassifier(max_depth=3, random_state=42)  
modelo.fit(XEnt, yEnt)
```

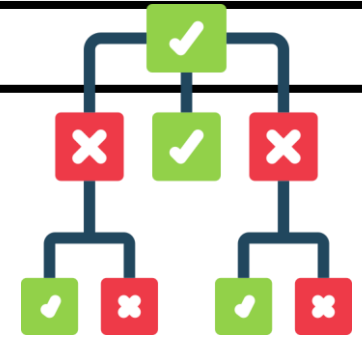
```
yPred = modelo.predict(XTst) # Predicción  
print("Exactitud del modelo:", accuracy_score(yTst, yPred))
```

👉 Exactitud del modelo: 1.0

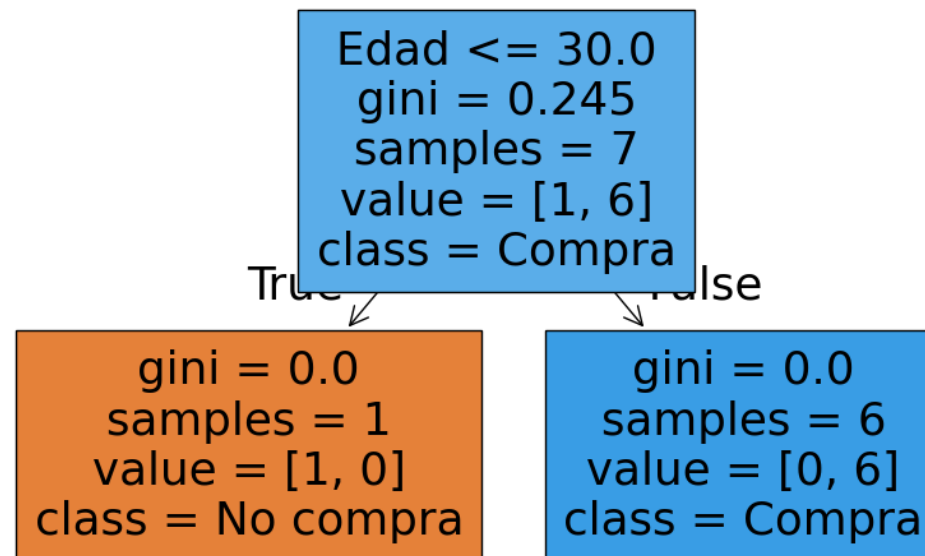


Árboles de decisión

```
# Visualización del árbol
plt.figure(figsize=(10,6))
plot_tree(modelo, feature_names=['Edad', 'Ingresos'],
          class_names=['No compra', 'Compra'], filled=True)
plt.title("Árbol de decisión - Compra según Edad e Ingresos")
plt.show()
```



Árbol de decisión - Compra según Edad e Ingresos



Árboles de decisión

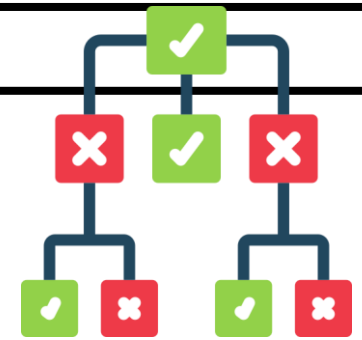
Sólo tiene 2 niveles; para porque es un Gini bajo (0) en las hojas
(Máxima Pureza)

Es decir que la columna Ingresos no es muy determinante

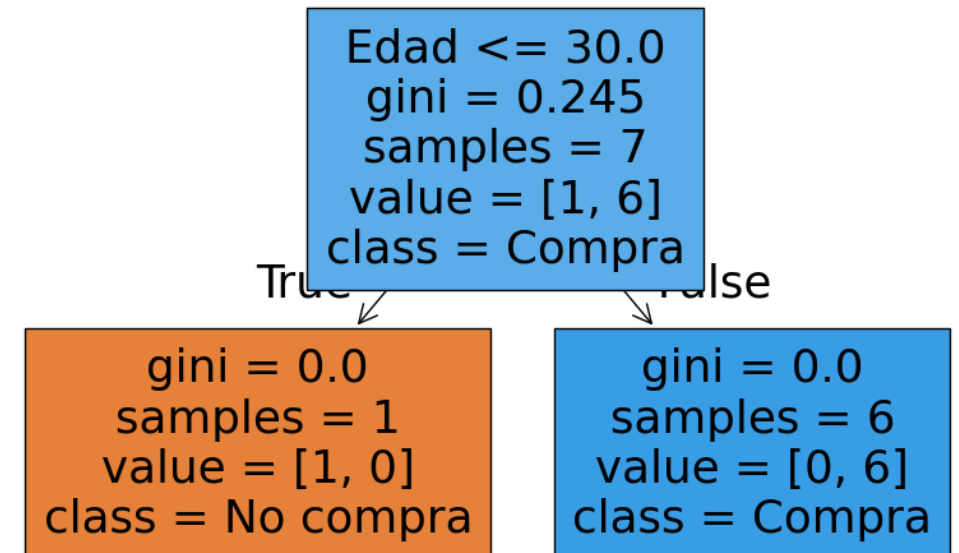
samples me dice cuántos casos llegaron a ese nodo (son 10 y 3 quedaron para testing)

value me dice la cantidad de casos que tenían cada clase (classifier)

class determina respuesta (mayoría) si el caso no avanza en el árbol y produce respuesta allí

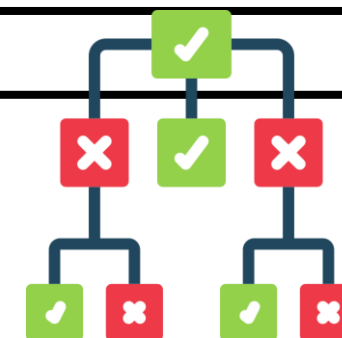
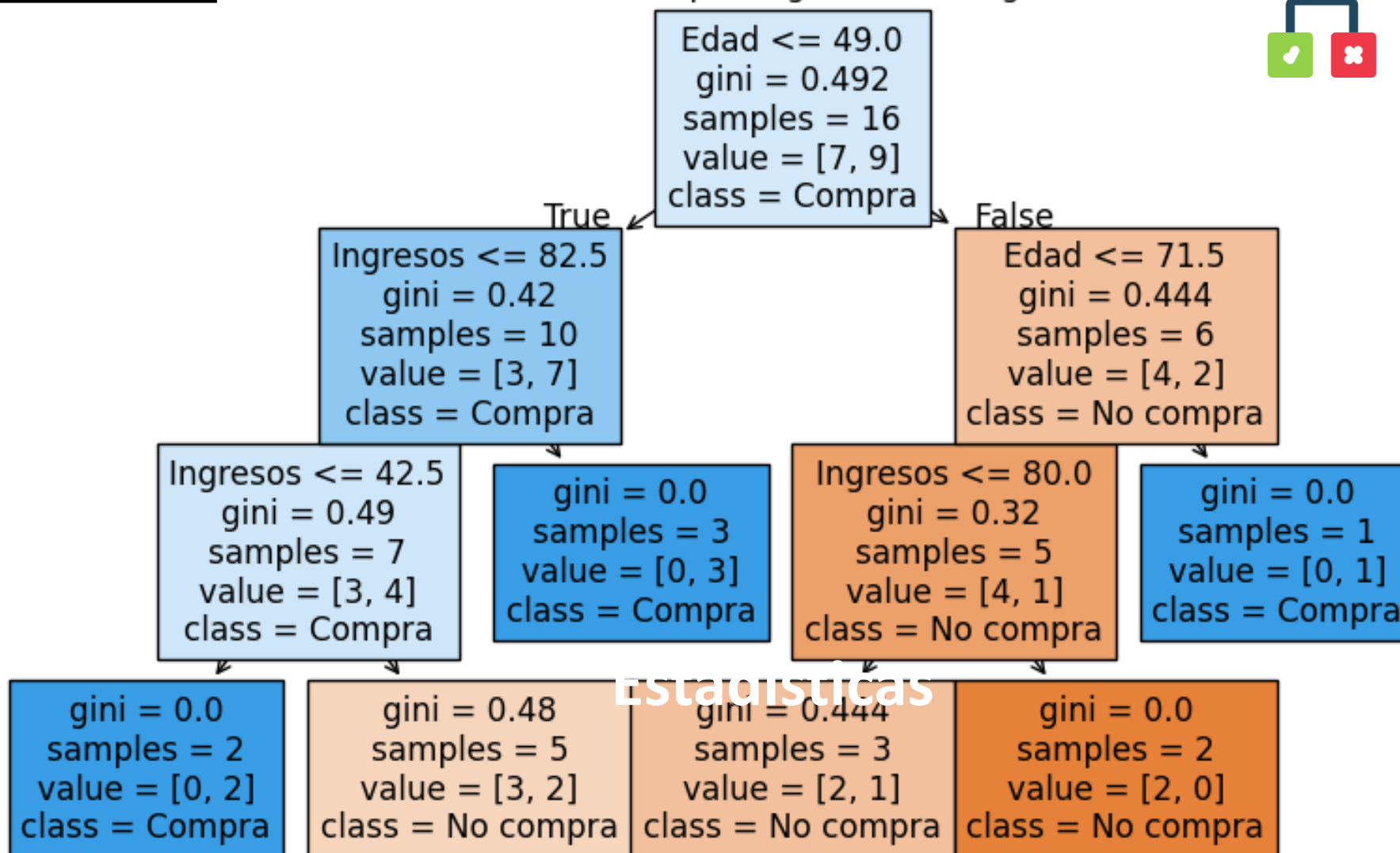


Árbol de decisión - Compra según Edad e Ingresos



Árboles de decisión

Árbol de decisión - Compra según Edad e Ingresos



Árboles de decisión

Original

Edad Ingreso Compra

25	40	0
30	20	0
45	80	1
35	60	1
50	90	1
23	25	0
40	100	1
60	70	1
22	15	0
48	85	1

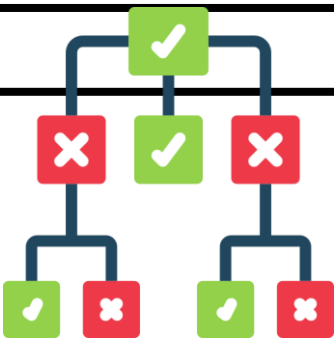
Nuevo

Edad Ingreso Compra

25	40	0
30	20	0
35	80	1
40	25	1
55	70	1
80	17	1
35	60	0
50	90	0
23	25	1
57	79	0
48	110	1
63	150	0

Edad Ingreso Compra

40	100	1
60	70	0
52	15	0
27	78	1
29	1000	1
40	55	1
48	85	1
18	15	1
22	65	1
25	60	0
33	80	0





Bosque aleatorio o Random Forest (RF)

Combina muchos árboles de decisión para hacer predicciones más precisas y estables. En vez de entrenar 1 solo árbol, crea decenas o cientos, literal: un bosque

- ✓ Usa datos aleatorios para cada árbol. Cada árbol se entrena con una muestra aleatoria de los datos con reemplazo (Bootstrap) y un subconjunto aleatorio de las columnas (cada árbol ve solo parte de la información). **Así cada árbol es diferente**
- ✓ Cada árbol hace su propia predicción (clasificación/regresión)
- ✓ El modelo elige la respuesta más común (mayoría, si es clasificación; promedio si es regresión)

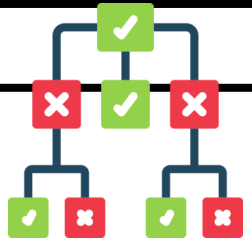
Ventajas:

- ✓ Puede manejar tanto datos numéricos como categóricos y mezcla
- ✓ Disminuye el sobreajuste. Tiene buena precisión sin mucho ajuste (al generar repetición de casos con Bootstrap termina diluyendo la incidencia de outliers o casos especiales)

Permite saber qué variables son más importantes

Desventajas:

- ✓ Es más lento que un solo árbol
- ✓ Menos interpretable. Es difícil explicar **por qué** tomó una decisión
- ✓ El modelo ocupa más memoria



Bosque aleatorio o Random Forest (RF)

Ejemplo Python: Predecir si una persona comprará o no un producto

```
from sklearn.ensemble import RandomForestClassifier # RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
# Datos: Edad (años), Ingresos (en miles de $)
```

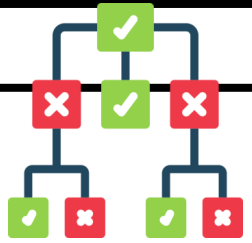
```
X = [[25, 30], [40, 80], [20, 20],
      [35, 70], [50, 90], [23, 25],
      [45, 85], [60, 95], [22, 15],
      [48, 75]]
```

```
y = [0, 1, 0, 1, 1, 0, 1, 1, 0, 1] # Etiquetas: 1 = compra, 0 = no compra
```

```
# División en entrenamiento y prueba
```

```
XEnt, XTst, yEnt, yTst = train_test_split(X, y, test_size=0.3, random_state=42)
```

 **Sigue...**



Bosque aleatorio o Random Forest (RF)

Ejemplo Python: Predecir si una persona comprará o no un producto

Creación y entrenamiento del modelo

```
modelo = RandomForestClassifier(n_estimators=100, random_state=42) #100 árboles
modelo.fit(XEnt, yEnt)
```

```
yPred = modelo.predict(XTst) # Predicción
```

```
print("Exactitud del modelo:", accuracy_score(yTst, yPred)) # Evaluación
```

Importancia de las variables/características/atributos/columnas

```
importancias = modelo.feature_importances_
```

```
caracteristicas = ['Edad', 'Ingresos']
```

```
for nombre, importancia in zip(caracteristicas, importancias):
```

```
    print(f"Importancia de {nombre}: {importancia:.2f}")
```

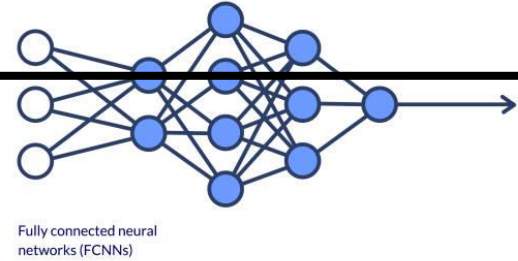
```
'''
```

zip es como map() y filter() arma una tabla tomando las

listas que le paso como las columnas

```
'''
```

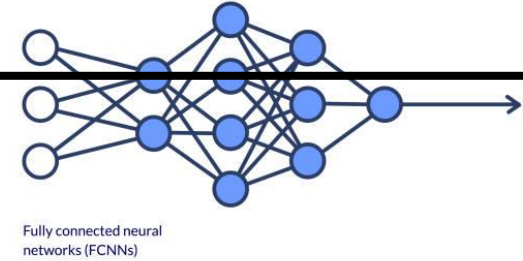
```
Exactitud del modelo: 1.0
Importancia de Edad: 0.52
Importancia de Ingresos: 0.48
```



Redes neuronales

Los algoritmos de aprendizaje profundo, como las redes **neuronales convolucionales** (CNNs) y las **redes neuronales recurrentes** (RNNs), son modelos supervisados altamente adaptables

Puedes utilizar estos modelos de ML para una variedad de tareas de aprendizaje supervisado, como la clasificación de imágenes y el procesamiento de lenguaje natural



Redes neuronales (RN) ¿Qué son y cómo se construyen?

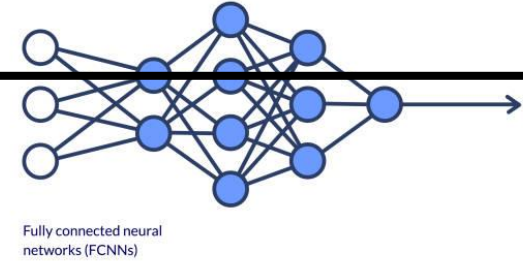
Una red neuronal es un modelo matemático inspirado en el modelo de funcionamiento del cerebro humano, que sirve para reconocer patrones y hacer predicciones

Estructura de neuronas en capas. Cada **neurona** toma números de entrada, aplica una función matemática y pasa el resultado a la siguiente capa

Al final, la red da una salida, ej: **Esto es un gato** o **La casa vale \$200,000**

¿Cómo se modela una RN?

1. Se recolecta información
2. Se modela definiendo las capas y neuronas
3. Se inicializan aleatoriamente pesos para cada neurona
4. Se ajustan los pesos para mejorar los caminos
5. Se usa el modelo entrenado (paso anterior) para predecir nuevos datos



Redes neuronales (RN) ¿Cuándo usarlas?

Ventajas:

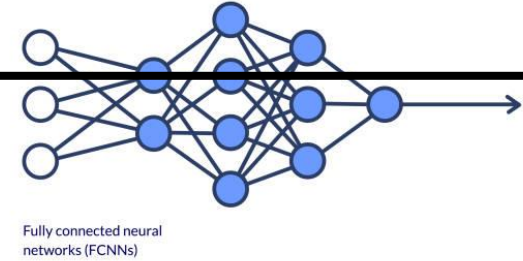
Aprenden patrones complejos Son muy buenas para tareas donde hay relaciones no obvias entre los datos, como imágenes, voz o lenguaje

Autonomía en el aprendizaje No se necesita programar reglas para los atajos, la red aprende a partir de los datos

Alta precisión Cuando se entrenan con muchos datos, pueden superar a otros modelos tradicionales

Capacidad de generalización Si se entrenan bien, pueden hacer buenas predicciones sobre nuevos datos

Flexibilidad Se puede usar para clasificación, regresión, detección de objetos, traducción, etc



Redes neuronales (RN) ¿Cuándo usarlas?

Desventajas:

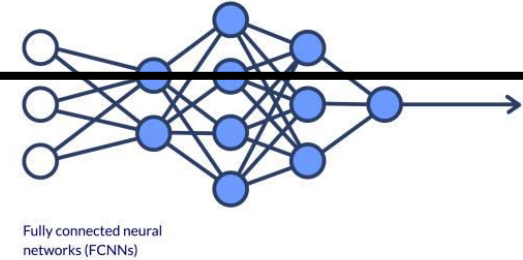
Necesitan muchos datos Para funcionar bien Requieren grandes cantidades de datos etiquetados para su entrenamiento

Consumen muchos recursos Entrenar redes grandes puede ser lento y requerir mucha memoria o capacidad de proceso

Difíciles de ajustar (tuning) Hay que elegir muchas cosas: número de capas, neuronas, tasa de aprendizaje, etc.

Poco interpretables Son una **caja negra**; es difícil entender por qué tomaron una decisión.

Pueden sobreajustar, si se entrena demasiado Puede aprender de memoria los atajos y no generalizar bien.



Redes neuronales (RN) ¿Cuándo usarlas?

Usarlas cuando:

Se tienen muchos datos

Los patrones son complejos (como imágenes, audio, texto)

Se quiere obtener alta precisión

✗ No usarlas cuando:

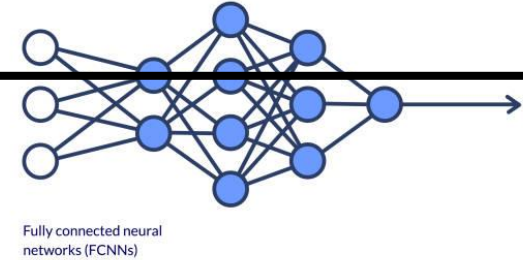
Se tiene pocos datos

Se necesita una explicación clara del resultado

Otro modelo más sencillo aplica

👉 En Python usar librería keras
en tensorflow

👉 Normalmente se usan para:
Reconocimiento de imágenes
Interpretación/Traducción de
Lenguaje Natural



Redes neuronales (RN) Ejemplo en Python

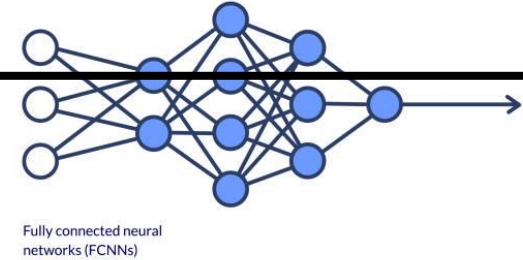
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

comentarios = ["Excelente atención", "Muy satisfecho con el servicio",
               "Todo funcionó perfecto", "La atención fue mala",
               "No resolvieron mi problema", "Experiencia horrible", "Estoy muy contento",
               "Me ayudaron rápidamente", "Pésimo servicio", "No volvería a contratar",
               "Nada mal", "No tan bueno como esperaba", "Flojo",
               "volvería a contratarlos", "Buenísimo"]

y = [1,1,1,0,0,0,1,1,0,0,1,0,0,1,1]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(comentarios) #comentarios se traducen a números
```

 Sigue...



Redes neuronales (RN) Ejemplo en Python

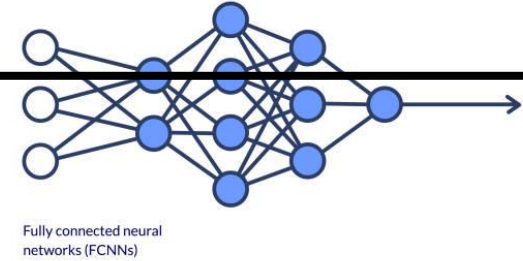
```
XEnt, XTst, yEnt, yTst = train_test_split( X,y, test_size=0.2,  
      random_state=42)
```

```
modelo = MLPClassifier(  
    hidden_layer_sizes=(10,), #cant capas intermedias  
    activation='relu', #apaga neuronas que no aportan en las capas intermedias  
        # relu es el actual estándar  
    solver='adam', #algoritmo de reasignación de pesos  
    max_iter=1000, #chico no aprende, grande sobreajusta  
    random_state=42)
```

```
modelo.fit(XEnt, yEnt)  
pred = modelo.predict(XTst)  
print("Accuracy:", accuracy_score(yTst, pred))
```

```
nuevo = ["Atención excelente", "Desastre", "No tan malo", "Realmente bueno"]  
nuevoVec = vectorizer.transform(nuevo)  
print(nuevo)  
print(modelo.predict(nuevoVec))  
print(modelo.predict_proba(nuevoVec))
```





Redes neuronales (RN) Ejemplo en Python

```
['Atención excelente', 'Desastre', 'No tan malo', 'Realmente bueno']
```

```
[0 1 0 1]
```

Malo

Bueno

```
[[0.77428886 0.22571114]
```



```
[0.41865103 0.58134897]
```

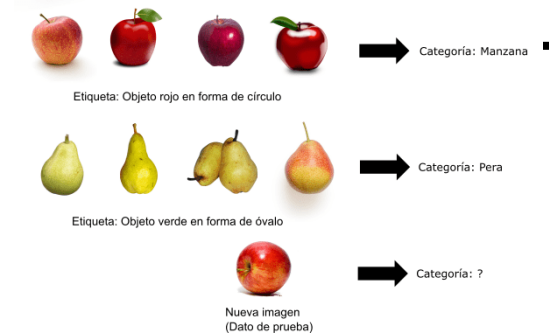


```
[0.76715107 0.23284893]
```



```
[0.41868188 0.58131812]]
```

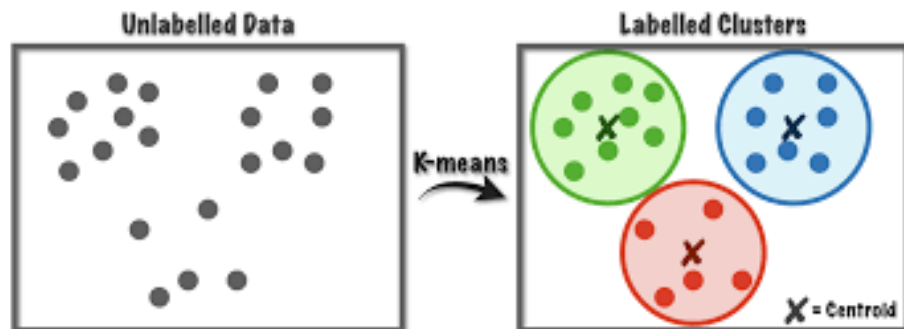


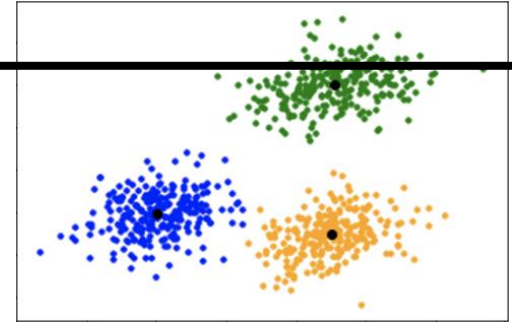


Modelos de aprendizaje no supervisado

El aprendizaje no supervisado es un tipo de aprendizaje automático en el que el algoritmo investiga patrones y estructuras en los datos sin producir una salida etiquetada

Estos métodos intentan encontrar patrones inherentes o correlaciones en los datos en lugar de predecir etiquetas específicas

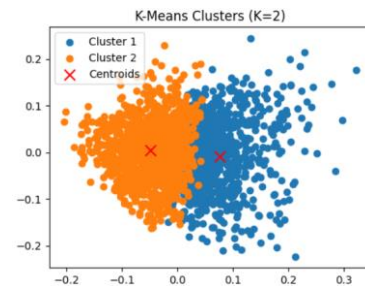


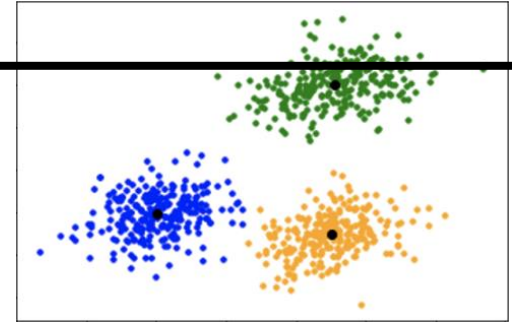


Agrupamiento K-Means

Método de clustering popular que divide los datos en grupos basados en similitudes

Busca reducir la varianza dentro de los grupos asignando puntos de datos al centro del clúster de forma iterativa





Agrupamiento K-Means ¿Cómo funciona K-Means?

Elige el número de grupos (k)

Coloca k **centros** iniciales aleatoriamente

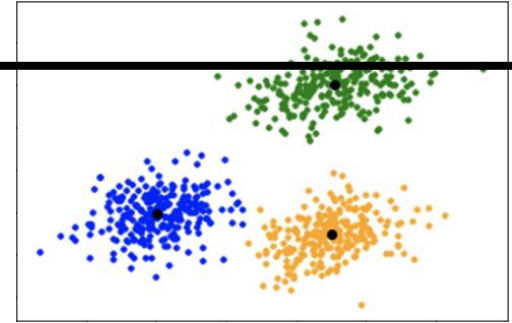
Cada punto se asigna al centro más cercano (formando grupos)

Se mueven los centros al centro de sus grupos

Repite hasta que los centros no cambien mucho

¿Para qué es bueno?

- ✓ Agrupar clientes por comportamiento de compra
- ✓ Agrupar colores similares en una imagen
- ✓ Agrupar datos genéticos, textos, etc.



Agrupamiento K-Means ¿Cómo funciona K-Means?

Ejemplo Python: Agrupar puntos en el plano en dos grupos

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
# Creamos algunos puntos de ejemplo
```

```
X = np.array([[1, 2], [1.5, 1.8], [5, 8], [8, 8],
              [1, 0.6], [9, 11], [8, 2], [10, 2], [9, 3]])
```

```
kmeans = KMeans(n_clusters=2) # Creamos modelo para 2 grupos
kmeans.fit(X) # Entrenamos el modelo
```

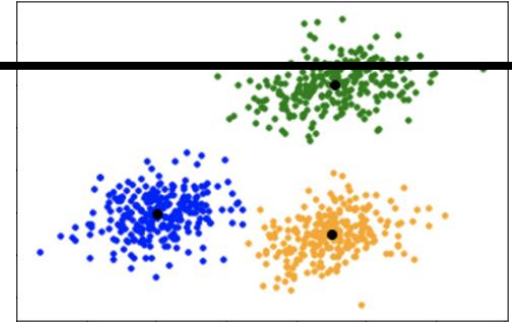
```
# Obtenemos predicciones: a qué grupo pertenece cada punto
```

```
grupos = kmeans.labels_
```

```
# Obtenemos los centros de los grupos
```

```
centroids = kmeans.cluster_centers_
```

 Sigue...



Agrupamiento K-Means ¿Cómo funciona K-Means?

Ejemplo Python: Agrupar puntos en el plano en dos grupos

```
# Graficamos
```

```
colores = ["g.", "r."] # Verde y rojo para los dos grupos
```

```
for i in range(len(X)):
```

```
    plt.plot(X[i][0], X[i][1], colores[grupos[i]], markersize=10)
```

```
# Dibujamos los centros de los grupos
```

```
plt.scatter(centroids[:, 0], centroids[:, 1], marker="x", s=150,  
            c='black')
```

```
plt.title("Agrupamiento con K-Means")
```

```
plt.xlabel("X")
```

```
plt.ylabel("Y")
```

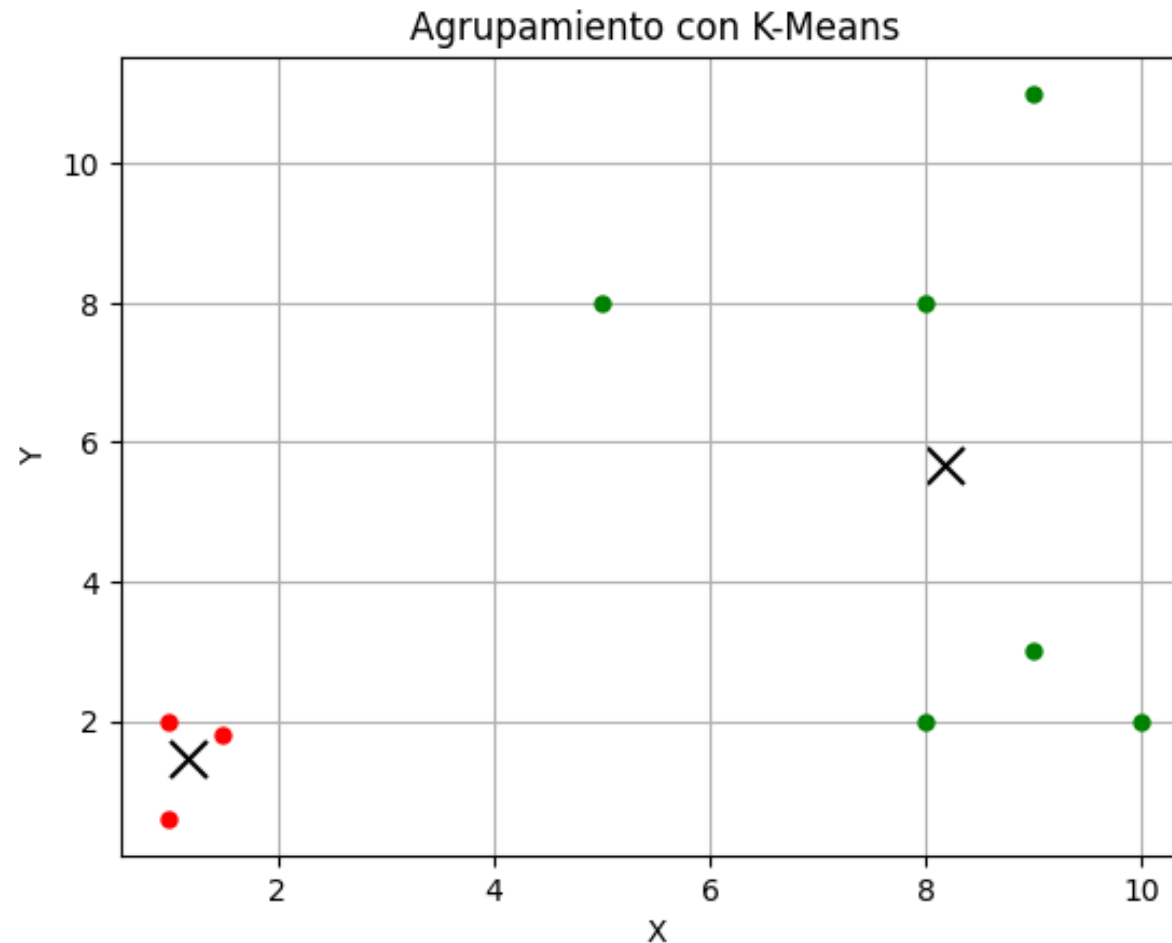
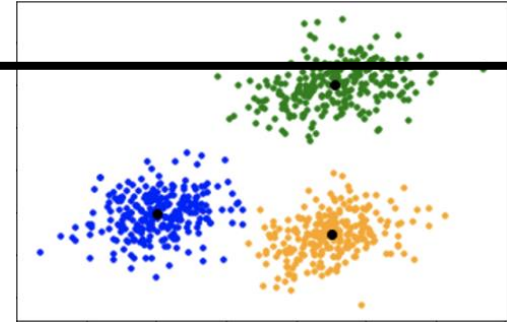
```
plt.grid(True)
```

```
plt.show()
```

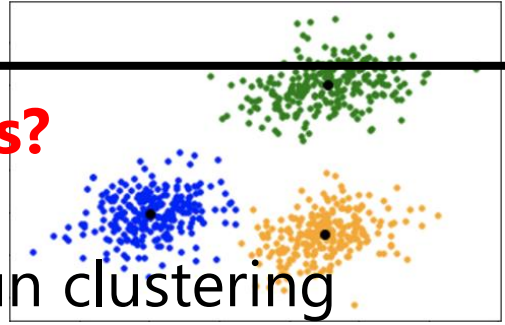
 Salida

Agrupamiento K-Means ¿Cómo funciona K-Means?

Ejemplo Python: Agrupar puntos en el plano en dos grupos



¿Cómo puedo evaluar qué tan buena es mi separación en clusters?



El **coeficiente Silhouette** es una métrica para evaluar la calidad de un clustering

Un buen cluster tiene puntos muy parecidos entre sí y muy diferentes de los puntos de otros clusters

El **coeficiente Silhouette** compara la cohesión interna de un cluster con su separación respecto de los demás clusters

Valores cercanos a 1 indican grupos compactos y bien separados;

valores cercanos a 0 indican observaciones en zonas de transición;

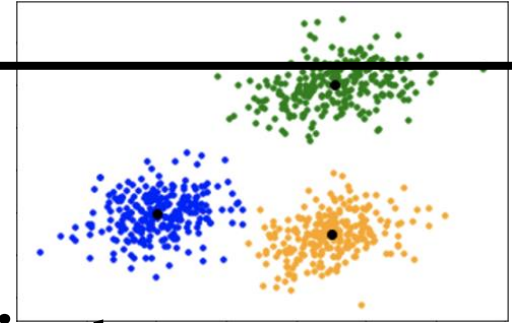
valores negativos sugieren asignaciones incorrectas

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Dde:

a(i): distancia media de i al resto de los puntos del cluster

b(i): distancia media entre i y todos los puntos del cluster más cercano



Se usa habitualmente para elegir un buen k , calculando el **coeficiente Silhouette promedio** en cada caso y usando el k que lo maximiza

Atención!

Es sensible a la función de distancia

Lo que para Euclides puede ser un buen negocio, para Manhattan, Coseno, etc. Puede ser una mala división

Modelos de Aprendizaje por Refuerzo

El aprendizaje por refuerzo es un sistema de aprendizaje automático en el que un agente aprende a tomar decisiones interactuando con su entorno

El agente aprende una política que optimiza las recompensas acumulativas con el tiempo al recibir información en forma de recompensas o penalizaciones



Q-Learning

Algoritmo de aprendizaje por refuerzo sin modelo que ayuda a los agentes a aprender la mejor política de selección de acciones

Mantiene una tabla Q que almacena las recompensas acumulativas esperadas para cada par estado-acción (la tabla Q es el modelo...)

Redes neuronales profundas con Q (DQN)

DQN es una extensión de Q-Learning que utiliza redes neuronales profundas para aproximar los valores de Q

Ha demostrado ser efectivo en la resolución de tareas complejas

<https://www.knowmadmood.com/blog/alphazero-la-revolucin-del-machine-learning>

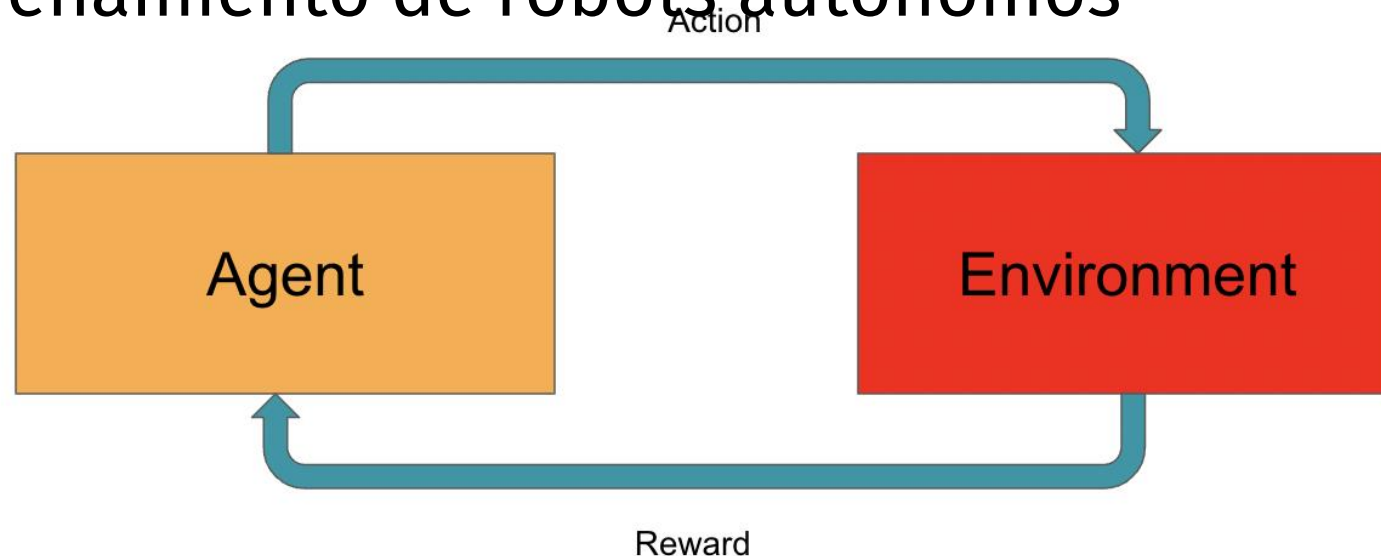
SARSA (State-Action-Reward-State-Action)

SARSA, al igual que Q-Learning, es un algoritmo de aprendizaje por refuerzo sin modelo

Determina la mejor política al estimar los valores de Q para pares estado-acción y emplear modificaciones en la política

A diferencia de los anteriores tiene un marco de políticas definidas

Ideal para entrenamiento de robots autónomos



Una vez que se creó y entrenó un modelo es importante evaluar su rendimiento en un conjunto de testeo

Si el modelo fue subajustado será muy fácil de apreciar, pero no así el caso de sobreajuste
Para evaluar esta última posibilidad conviene probar el modelo con un conjunto de datos diferente al de entrenamiento

Se pueden crear diferentes modelos (con reducción de dimensionalidad o no) y verificar su rendimiento en término de su capacidad de acierto

